

Correcting the Design of Bulk Execution

P2181r1

Jared Hoberock (jhoberock@nvidia.com)

Michael Garland (mgarland@nvidia.com)

November 13, 2020

1. Introduction

A bulk execution interface was introduced as a fundamental operation supported by executors in [N4406](#) (“Parallel algorithms need executors”) and adopted in [P0443r0](#), the first unified executor proposal, in the form of a `bulk_execute` interface. This interface has been present in P0443 from the beginning because a properly designed `bulk_execute` interface accomplishes two goals of fundamental importance. It provides the basis for exploiting platforms that support efficient mechanisms for creating many execution agents simultaneously, and it encapsulates the (potentially platform-specific) means of doing so.

The design of P0443 has evolved significantly since its initial revision, most notably to adopt the sender/receiver approach for lazy execution. The design of `bulk_execute` has lagged behind these changes, and is presented with inconsistent signatures in [P0443r14](#). The lack of a consistently defined interface for bulk execution must be resolved before P0443 can be adopted.

In this paper, we propose a design for bulk execution that corrects this defect in P0443r14. Our proposal:

- Defines `bulk_execute` as an interface for eager work submission, following the semantics of `execute`.
- Introduces a new `bulk_schedule` that provides a basis for lazy work submission, following the semantics of `schedule`.

Adopting these proposals requires only minor changes to P0443. They do not change any of the concepts or mechanisms in P0443 aside from the defective definition of `bulk_execute`. They also broaden the scope of bulk execution by providing for both eager and lazy submission, rather than eager submission alone.

1.1. Changes in Revision 1

The following are the primary changes made in this revision of the paper:

- Adopted the interface proposed in [P2224](#) (“A better `bulk_schedule`”), which supercedes the `bulk_schedule` interfaces explored in both [Revision 0](#) and [P2209](#), while also eliminating the need for any form of `bulk_join`.

- Eliminated the `many_receiver_of` concept, which is no longer needed in the revised design of `bulk_schedule`.
- Specified that `connect` is called in each agent of a launch created by `bulk_schedule`.
- Consistent with ongoing discussions in the design review of P0443r14, we have distinguished executors and schedulers more carefully and do not assume implicit conversion between them.
- The default implementation of `bulk_execute` now makes a single call to `execute`.
- Replaced types `executor_shape_t<E>` and `executor_index_t<E>` with a single type `executor_coordinate_t<E>`.
- Added a new discussion section on a possible refactoring of `bulk_schedule` into two operators.

The bulk of the changes occur in Section 3 (*Corrected Bulk Interface*) and Section 5 (*Discussion*).

2. Background

Every revision of P0443 has included `bulk_execute` as the lowest level primitive operation for creating work in bulk through an executor. Both P0443 and the interface of `bulk_execute` have evolved since its first revision, but the intended functionality of `bulk_execute` has remained unchanged: it is the basis for creating a group of function invocations in bulk in a single operation.

The design sketched in [P1660r0](#) (“A compromise executor design sketch”) is the basis for the current specification in [P0443r14](#). While reaffirming the importance of bulk execution, it proposed only to:

┆ Introduce a customizable bulk execution API whose specific shape is left as future work.

Section 5.3 of that paper provided some “highly speculative” suggestions, but no definitive design was given. P0443r14 also attempts to incorporate the proposal of [P1993r1](#) (“Restore shared state to `bulk_execute`”) to return a sender result so that dependent work may be chained with a bulk task.

This results in the intended interface of `bulk_execute` in P0443r14:

```
sender_of<void> auto bulk_execute(executor auto ex,
                                invocable auto f,
                                executor_shape_t<decltype(ex)> shape);
```

This formulation creates `shape` invocations of function `f` on execution agents created by executor `ex`. A sender of `void` corresponding to the completion of these invocations is the result.

2.1. Inconsistent definitions in P0443

Despite this intent, the material addressing bulk execution in [P0443r14](#) is not self-consistent. This inconsistency is particularly apparent in the envisioned return type of `bulk_execute`.

- Section 1.3 includes an example use of `bulk_execute` that returns a sender:

```
sender auto s = execution::bulk_execute(ex, ...);
```

- Section 2.2.3.9 specifies the customization point `execution::bulk_execute`, yet remains silent on its return type.
- Section 2.5.5.5 specifies that the interface of `static_thread_pool` includes a `bulk_execute` method returning `void`:

```
template<class Function>
void bulk_execute(Function&& f, size_t n) const;
```

Our proposal eliminates this inconsistency with a single, clearly defined interface for `bulk_execute`.

2.2. Shared state and dependent tasks

Programs need to chain dependent tasks together, in both the singular and bulk cases. Furthermore, it is particularly important to provide a means for delivering shared state (e.g., barrier objects or shared output arrays) to all the constituent invocations of a bulk operation.

SG1 considered this issue at its February 2020 meeting in Prague, and decided that:

Poll: We should add a sender argument and sender result to bulk execution functions (providing an opportunity to build shared state, established dependencies in/out)

SF	F	N	A	SA
17	7	0	0	0

Our proposal fulfills this requirement with a new `bulk_schedule` interface.

3. Corrected Bulk Interface

The inconsistent interfaces for bulk execution in [P0443r14](#) arise from uncertainty about the means for integrating senders into the `bulk_execute` interface. The design for singular execution in P0443r14 avoids this confusion by providing *two* interfaces (`execute` and `schedule`) that disentangle the concerns of eager submission and lazy scheduling. The defects in the interface for bulk execution in P0443r14 are readily corrected by adopting a similar approach.

3.1. Design Synopsis

The `bulk_execute` operation should be the mechanism for eager submission of work in bulk, a role analogous to `execute`. The interface sketched in P0443 should be replaced with one of the following form:

```
template<executor E, invocable F>
void bulk_execute(E ex,
                 F&& f,
                 executor_coordinate_t<E> shape);
```

The invocable `f` accepts a single argument corresponding to its assigned coordinate in `shape`. The work to be done by this invocation has been submitted for execution in a group of the given `shape` before `bulk_execute` returns, but the point at which actual execution occurs is implementation defined. This interface provides no

further mechanism for synchronizing with the completion of the submitted work. Therefore, some additional means of synchronization is required to determine when the bulk operation has completed. Such facilities could be provided by executor-specific interfaces or sequencing guarantees. In general, it is necessary to use a synchronization object such as the `std::barrier` used in the following example.

```
auto exec = ...
vector<int> ints = ...
barrier bar(ints.size() + 1);

// launch work to mutate a vector of integers
bulk_execute(exec,
             [&](size_t idx) { ints[i] += 1; bar.arrive(); },
             ints.size());

// wait for work to be completed
bar.arrive_and_wait();
```

A new interface is required for scheduling work for later submission. This interface should use senders as the means of composition and for ordering chains of dependent operations. This is the role of `schedule` for singular execution; therefore, we propose the addition of an analogous bulk operation. This new `bulk_schedule` operation should have an interface of the following form:

```
template<scheduler S, invocable F, class... Ts>
sender_of<Ts...> bulk_schedule(sender_of<Ts...> auto&& prologue,
                              S sched,
                              executor_coordinate_t<S> shape,
                              F&& factory);
```

which was proposed in [P2224](#) and adopted by SG1 at its meeting on October 12, 2020.

The returned object is a sender representing the entire computation of the bulk section. The invocable `factory` is responsible for constructing a sender that represents the computation to be performed in each agent of the bulk launch. The signature for this is a sender-factory and should be of the form:

```
auto factory(sender_of<executor_coordinate_t<S>, Ts&...>) ->
sender_of<void>
```

The factory is called with a single parameter: a sender representing the initiation of each agent. This sender delivers to its receiver both an agent index and the values (if any) provided by `prologue`. The factory must return a `sender_of<void>` representing the computation to be performed by each agent. This bulk operation will be submitted for execution when the sender returned by `bulk_schedule` is connected to a receiver and started. In each agent of the bulk launch, a receiver will be connected to the sender returned by the factory, or a copy thereof. The resulting operation returned by `connect` will subsequently be executed in that agent with `start`.

The “prologue” sender provided to `bulk_schedule` is intended to deliver state that should be shared across the group of execution agents created upon execution. Each agent is identified by an index sent via `set_value` along with the shared state (if any) delivered by the prologue. The following example illustrates

the use of `bulk_schedule`, along with functionality proposed in [P1897r3](#), to share a collection of integers across a group of execution agents and mutate each element individually.

```
auto sched = ...
std::vector<int> ints = ...

// assemble a computation to mutate a vector of integers
auto increment =
    bulk_schedule(just(ints),
                  sched,
                  ints.size(),
                  transform([](size_t idx, std::vector<int>& ints)
                          {
                              ints[idx] += 1;
                          }));

// perform the computation and wait for its completion
execution::sync_wait( increment );
```

Because `increment` is a sender, we can wait on its completion using a generic `sync_wait` operation rather than having to embed a synchronization object like a barrier directly in the code.

3.2. Specification of `bulk_execute`

[Editorial note: Replace Section 2.2.3.9 (`execution::bulk_execute`) in P0443r14 with the material in this section. —end editorial note]

The name `execution::bulk_execute` denotes a customization point object. If:

```
is_convertible_v<decltype(S),
execution::executor_coordinate_t<decltype(remove_cvref_t<E>>>
```

is true, then the expression `execution::bulk_execute(E, F, S)` for some subexpressions `E`, `F`, and `S` is expression-equivalent to:

- `E.bulk_execute(F, S)`, if that expression is valid. If the function selected does not execute `F` in an `S`-shaped group of execution agents with forward progress `query(E, execution::bulk_guarantee)` on executor `E`, the program is ill-formed with no diagnostic required.
- Otherwise, `bulk_execute(E, F, S)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void bulk_execute();
```

and that does not include a declaration of `execution::bulk_execute`.

If the function selected does not bulk execute `F` with shape `S` on executor `E`, the program is ill-formed with no diagnostic required.

- Otherwise, if the type of `E` models `executor`, and the type of `F` and `executor_coordinate_t<remove_cvref_t<E>> model invocable`, and if `query(E, execution::bulk_guarantee)` equals `execution::bulk_guarantee.unsequenced`
 - If the type of `F` models `copy_constructible`, then equivalent to `execution::execute(E, [f=DECAY_COPY(F)]{ for(auto idx=0; idx<S; ++idx) invoke(f, idx); })`.
 - Otherwise, equivalent to `execution::execute(E, [&]{ for(auto idx=0; idx<S; ++idx) invoke(F, idx); })`.

[*Note:* The requirement of `bulk_guarantee.unsequenced` here means that the default implementation is not available to an executor that chooses to advertise a different guarantee. Such executors are required to provide implementations of `bulk_execute` that fulfill the advertised guarantee. —end note]

- Otherwise, `execution::bulk_execute(E, F, S)` is ill-formed.

3.3. Specification of `bulk_schedule`

[*Editorial note:* Introduce a new Section 2.2.3.10 (`execution::bulk_schedule`) containing the material in this section. —end editorial note]

The name `execution::bulk_schedule` denotes a customization point object. For some subexpressions `scheduler`, `shape`, `prologue`, and `factory`: let `E` be a type such that `decltype((scheduler))` is `E`, and let `S` be a type such that `decltype((shape))` is `S`, and let `P` be a type such that `decltype((prologue))` is `P`, and let `F` be a type such that `decltype((factory))` is `F`. The expression `execution::bulk_schedule(prologue, scheduler, shape, factory)` is ill-formed if `typed_sender<P>` is not true.

Otherwise, the expression `execution::bulk_schedule(prologue, scheduler, shape, factory)` is expression-equivalent to:

- `scheduler.bulk_schedule(prologue, shape, factory)`, if that expression is valid and its type `R` satisfies `typed_sender<R>`, and if `sender_traits<R>::value_types<tuple, variant>` is `variant<tuple<executor_coordinate_t<decltype(scheduler)>, add_lvalue_reference_t<Values>...>...>` for all `Values...` parameter packs sent by `prologue`.
- Otherwise, `bulk_schedule(prologue, scheduler, shape, factory)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
void bulk_schedule();
```

and that does not include a declaration of `execution::bulk_schedule`, and if that expression's type satisfies `typed_sender<R>`, and if `sender_traits<R>::value_types<tuple, variant>` is `variant<tuple<executor_coordinate_t<decltype(scheduler)>, add_lvalue_reference_t<Values>...>...>` for all `Values...` parameter packs sent by `prologue`.

- Otherwise, if `scheduler<E>` is true and `executor_coordinate_t<E>` is `S`, returns a sender object `s` whose implementation-defined type `R` satisfies `typed_sender<R>`.
`execution::connect(s, r)` returns an object `o` whose implementation-defined type satisfies `operation_state`.
 - Let `values...` be a parameter pack of values sent by `prologue`, and `coord` be a coordinate spanned by `shape`. Let `__just(args...)` be an exposition-only operation that sends the parameter pack `args...` to a connected receiver, and let `__discard_receiver` be an exposition-only receiver whose `set_value`, `set_error`, and `set_done` functions have no effect. `execution::start(o)` calls `submit(factory(__just(coord, values...), __discard_receiver{}))` once for each `coord` spanned by `shape`. Upon completion of all such invocations, it calls `execution::set_value(move(r), values...)`.
 - Otherwise, let `error` be an error sent by `prologue`. `execution::start(o)` calls `execution::set_error(move(r), error)`.
 - Otherwise, `execution::start(o)` calls `execution::set_done(move(r))`.
- Otherwise, `execution::bulk_schedule(prologue, scheduler, shape, factory)` is ill-formed.

4. Supporting Definitions

Each instantiation of the body in a bulk launch receives an *index* argument that identifies that instance. The index space of the launch itself is described by a *shape* parameter provided to `bulk_execute` and `bulk_schedule`. The current draft of P0443 uses separate types for these values: `executor_index_t<E>` and `executor_shape_t<E>`, respectively. Our implementation experience to date suggests that there is no benefit to differentiating these types. Therefore, our proposal replaces them with a single `executor_coordinate_t<E>` type that is used for both purposes.

4.1. Definitions of execution

An editorial note in [P0334r14, Section 2.2.3.4](#) says that:

We should probably define what “execute the function object `F` on the executor `E`” means more carefully.

We suggest the following definition:

An executor *executes* an expression by scheduling the creation of an execution agent on which the expression executes. Invocable expressions are invoked by that execution agent. Execution of expressions that are not invocable is executor-defined.

Furthermore, we suggest adding the analogous definitions for bulk execution:

A *group of execution agents* created in bulk has a *shape*. Execution agents within a group are identified by *indices*, whose unique values are the set of contiguous indices spanned by the group’s shape.

An executor *bulk executes* an expression by scheduling the creation of a group of execution agents on which the expression executes in bulk. Invocable expressions are invoked with each execution agent’s index. Bulk

execution of expressions that are not invocables is executor-defined.

5. Discussion

The preceding sections contain the entirety of our proposed corrections and additions to [P0443r14](#). This section provides some additional background explanation and highlights some additional proposals that may be considered separately.

5.1. Design of the bulk interface

This proposal positions `bulk_execute` as the direct analogue of `execute`. Both are low-level interfaces for creating execution and are necessary to expose platform-level work creation interfaces, which may be implemented outside the standard library. Furthermore, individual executor types may provide important platform-provided forward progress guarantees, such as a guarantee of mutual concurrency among agents.

While the default implementation of the `bulk_execute` customization point decays to a loop in the absence of an executor-provided method, the `bulk_execute` operation is semantically distinct from a loop. Every loop construct in the standard is either explicitly sequential or permitted to fall back to a sequential equivalent at the sole discretion of the implementation. In contrast, executors may be used with `bulk_execute` to guarantee execution semantics that have no lowering onto sequential execution. For example, an executor whose `bulk_execute` method guarantees that all its created agents are concurrent with each other has no sequential equivalent.

5.2. Execution policies

As in all prior revisions of P0443, the `bulk_execute` interface we propose does not include an execution policy argument. The use of execution policies in `bulk_execute` would be fundamentally inconsistent with their use throughout the rest of the library.

Execution policies were designed as a mechanism for customizing the execution of algorithms in the standard library in a way that could support the broadest possible range of architectures (see [N3554](#)). As designed, they are suitable for customizing operations that can optionally change execution semantics (e.g., parallel execution in multiple threads). They are not, however, suitable for customizing low-level interfaces such as `bulk_execute` where mandatory execution semantics have already been specified in the form of an executor.

For every invocation of an algorithm with an execution policy, it is valid to replace the policy specified in the call with `execution::seq` without changing the meaning of the program. Similarly, conforming implementations are granted the freedom to fall back to sequential execution, regardless of the policy specified. This cannot be done with `bulk_execute` if the executor provides guarantees (e.g., non-blocking execution or concurrent forward progress) inconsistent with sequential execution in the calling thread.

The use of execution policies in the library is also designed to support a variety of vendor-supplied execution policies. Providing such vendor-specific policies to `bulk_execute` would typically have no meaning unless the executor is also a vendor-specific executor specifically designed to recognize that policy. In this case, all information provided by the policy could have been provided via the executor itself, making the policy parameter unnecessary. Once the executor semantics have been customized via the property-based `require` mechanism, any semantics implied by a policy are at best redundant and at worst contradictory.

5.3. Copying invocables

Both `execute`, and by extension `bulk_execute`, allow non-copyable invocable types. This manifests in the third bullet point of the specification of `bulk_execute`, which has two cases. The first case opportunistically creates copies of the user’s invocable when it is possible to do so. Each agent created by the executor receives one of these copies. Otherwise, if the invocable is not copyable, each agent receives a reference to the invocable instead of a copy. This policy was chosen to ensure that invocables containing non-copyable, non-moveable types (e.g., synchronization objects) are still usable with `bulk_execute`. The caller of `execute` and/or `bulk_execute` must ensure that a non-copyable, non-moveable invocable outlives the group of agents that invokes it and that overlapping invocations do not create data races.

5.4. Default implementations

We follow the existing practice in P0443 and specify default implementations for the `bulk_execute` and `bulk_schedule` customization points when the executor/scheduler does not provide corresponding methods. These default implementations are meant as a fallback only. Executors/schedulers that aim to support parallel execution of some form should provide their own implementations of the bulk interfaces.

It would be valuable if the default implementation of `bulk_schedule` could rely upon `bulk_execute`. This would encapsulate the details of submission in a single place, and it would help guarantee semantic equivalence between eager and lazy mechanisms for work submission. The current design of [P0443r14](#) prevents this because executors rely on exceptions for error reporting whereas schedulers rely on `set_error` calls on receivers. Bridging the gap between these two would likely result in a simpler and more seamless design. A candidate solution was sketched in [P1660, Section 5.2](#), which recommended allowing the caller of `execute` or `bulk_execute` to control the error delivery channel by providing either an invocable—resulting in the use of exceptions—or a receiver—resulting in delivery via `set_error`. A more complete solution is proposed in [P2254](#).

5.5. Additional convenience overloads

The `bulk_schedule` interface may be marginally more convenient if an additional overload is provided without a prologue sender:

```
template<scheduler S, invocable F, class... Ts>
sender_of<void> bulk_schedule(S sched,
                             executor_coordinate_t<S> shape,
                             F&& factory);
```

While an equivalent result can already be achieved by passing a suitable “empty” prologue sender through the interface we have proposed, this overload would be more convenient for the user of the interface.

It may also be worth considering adding an overload of `schedule` that accepts a prologue sender, mirroring the `bulk_schedule` interface we have proposed:

```
template<scheduler S, class... Ts>
sender_of<Ts...> schedule(sender_of<Ts...> auto&& prologue,
                          S sched)
```

Neither of these changes is essential, but adding these options to the existing overloads for `schedule` and `bulk_schedule` in P0443r14 and our proposal above, respectively, would make the scheduling interface more convenient and more predictable.

5.6. Possible refactoring of `bulk_schedule`

While implementing our proposed `bulk_schedule` interface, we discovered that our implementation could be simplified by decomposing the functionality of `bulk_schedule` into two primitive operations:

1. `on(prologue, scheduler)`
2. `bulk(prologue, shape, sender_factory)`

such that `bulk_schedule` can be formed from their composition:

```
template<typed_sender P, scheduler S, invocable SF>
typed_sender auto bulk_schedule(P&& prologue,
                               S scheduler,
                               scheduler_coordinate_t<S> shape,
                               SF sender_factory)
{
    return on(prologue, scheduler) | bulk(shape, sender_factory);
}
```

In our current implementation, the `on` operation establishes a transition from a sender chain’s “upstream” execution agent to a new “downstream” execution agent. The `bulk` operation shares values sent by its `prologue` sender across `shape` execution agents created by the `scheduler` associated with the `prologue`. We accomplish this by assuming that all `typed_senders` advertise their `scheduler` via a customization point `get_scheduler(sender) -> scheduler`.

Aside from the substantial reduction in implementation complexity, this refactoring has the side-benefit of providing the missing `scheduler` transition combinator `on`. If P0443 is revised to include our proposed `bulk_execute` and `bulk_schedule` interfaces only, the sole mechanism for midchain transition is by misuse of `bulk_schedule` as a particularly complicated form of `on`.

References

- Garland, Michael, Lee Howes, and Jared Hoberock. 2020. “A Better `bulk_schedule`.” <http://wg21.link/p2224>.
- Hoberock, Jared. 2020. “Restore Shared State to `bulk_execute`.” <http://wg21.link/p1993r1>.
- Hoberock, Jared, Michael Garland, and Olivier Giroux. 2015. “Parallel Algorithms Need Executors.” <http://wg21.link/N4406>.
- Hoberock, J., M. Garland, C. Kohlhoff, C. Mysen, C. Edwards, G. Brown, D. Hollman, et al. 2020. “A Unified Executors Proposal for C++.” <http://wg21.link/p0443r14>.
- Hoberock, J., M. Garland, B. Lelbach, M. Dominiak, E. Niebler, K. Shoop, L. Baker, L. Howes, D. Hollman, and G. Brown. 2019. “A Compromise Executor Design Sketch.” <http://wg21.link/p1660r0>.

Hoberock, J., J. Marathe, M. Garland, O. Giroux, V. Grover, A. Laksberg, H. Sutter, and A. Robison. 2013. “A Parallel Algorithms Library.” <http://wg21.link/N3554>.

Howes, Lee, Lewis Baker, Kirk Shoop, and Eric Niebler. 2020. “Bulk Schedule.” <http://wg21.link/p2209>.